Benchmarking Python, Dask, and Numba Parallelization Strategies for Pairwise Chi-Squared and *t*-Test Computation Against the NApy Package

Arafa Ferdous

Project report in MSc. in Artificial Intelligence
Direct supervisor: Fabian Woller
Supervising professor: Prof. Dr. David B. Blumenthal
Matriculation number: 23068624
Biomedical Network Science Lab, Department Artificial Intelligence in Biomedical Engineering, Friedrich-Alexander-Universität Erlangen-Nürnberg

Submission date: 28 July 2026

Abstract

We implement and benchmark four computational configurations: a naive Python reference implementation, a NumPy-vectorized implementation, a pure Dask-parallelized implementation, and a Dask+Numba-parallelized implementation, for two pairwise statistical tests commonly used in biomedical data analysis, the chi-squared test of independence and Student's *t*-test. Following the design of the NApy package [1], both tests operate on data matrices with pairwise missing-value handling. We benchmark runtime scaling with respect to the number of variables and the number of samples, and we investigate the effect of task granularity (workload per Dask process) on parallel runtime. Dask+Numba is consistently the fastest of our own implementations, but all four configurations remain 100–300× slower than NApy's compiled C++/OpenMP backend. Our workload experiments show that runtime improves by roughly 20–30× once scheduling overhead is amortized over sufficiently large per-task workloads, compared to the smallest workload sizes tested.

Contents

1 Introduction

Pairwise statistical testing, computing a test statistic and P-value for every pair of variables in a data matrix, is a common but computationally expensive operation in biomedical data analysis. This is especially true when working with datasets containing substantial amounts of missing data that must be handled on a pairwise basis rather than through complete-case deletion. As the number of variables grows, the number of required pairwise tests grows quadratically, making efficient implementation essential for interactive or large-scale analyses such as on-the-fly exploration of biomedical association networks [1].

NApy [1] is a recently published Python package that addresses this problem using a dual C++/OpenMP and Numba backend. The authors report runtime and memory improvements of up to three orders of magnitude over naive Python-based competitors, evaluated on simulated data and on the real-world CHRIS population cohort study. NApy formalizes pairwise missing-value handling as follows: given a data matrix $X \in \mathbb{R}^{F,S}$ of F features and S samples, and a designated missing-value sentinel m, the test for a feature pair (g, h) is computed only on the index set

$$\mathcal{I}(g, h) = \{(g_i, h_i) \mid g_i \neq m \wedge h_i \neq m\}. \tag{1}$$

Samples are pairwise-deleted rather than removed via complete-case analysis, which preserves more information in datasets with high missingness rates. We adopt exactly this missing-value handling strategy, defined in Equation (1), in all four of our implementations.

The NApy publication reports that existing Python tools such as NumPy, pandas, SciPy, and Pingouin each support only a subset of the required functionality. None of them combine pairwise missing-value deletion, P-value computation, effect-size calculation, and built-in parallelization for the chi-squared and t-tests we study here. However, the specific performance contributions of individual optimization strategies (vectorization, task-based parallelization via Dask, and just-in-time (JIT) compilation via Numba) are not disentangled in isolation in the original publication, which compares NApy directly against a single naive Python baseline.

We investigate this question directly: for two representative pairwise tests, how much runtime improvement does each optimization strategy contribute individually and in combination, and how does the result compare to NApy's fully compiled implementation? We further examine how Dask's task granularity, the number of variable pairs assigned to each parallel task, affects overall runtime, since this parameter is under the practitioner's control and represents a key tuning decision for real-world usage of task-based parallel frameworks. Our contribution is a controlled, step-by-step comparison of four self-implemented configurations, verified for correctness against both `scipy.stats` ground truth and NApy's own output. This provides a more granular view of the performance gap than a single-baseline comparison affords.

The remainder of this report is structured as follows. Section 2 presents our runtime results across problem size and Dask workload granularity. Section 3 discusses these findings in the context of the original NApy publication and outlines limitations and an outlook. Section 4 describes the statistical tests, the four implementations, and the benchmarking design in detail. Sections 5 and 6 provide code and data availability information.

2 Results

2.1 Runtime scaling with number of variables

Figures 1 and 2 show runtime as a function of the number of variables, with sample count fixed at 500. For both tests, the ranking is consistent across all tested sizes: the Python reference implementation is slowest throughout, followed by the vectorized NumPy implementation, then pure Dask and Dask+Numba (close to each other), with NApy fastest by a wide margin. At the largest tested size (300 features), the chi-squared test required 19.31 s (Python reference), 7.74 s (vectorized), 6.35 s (Dask),

6.60 s (Dask+Numba), and 0.11 s (NApy), a ∼182-fold difference between NApy and the naive reference. The *t*-test showed a similar pattern: 26.22 s (reference), 14.20 s (vectorized), 19.80 s (Dask), 17.59 s (Dask+Numba), and 0.19 s (NApy), a ∼139-fold difference (Table 1).

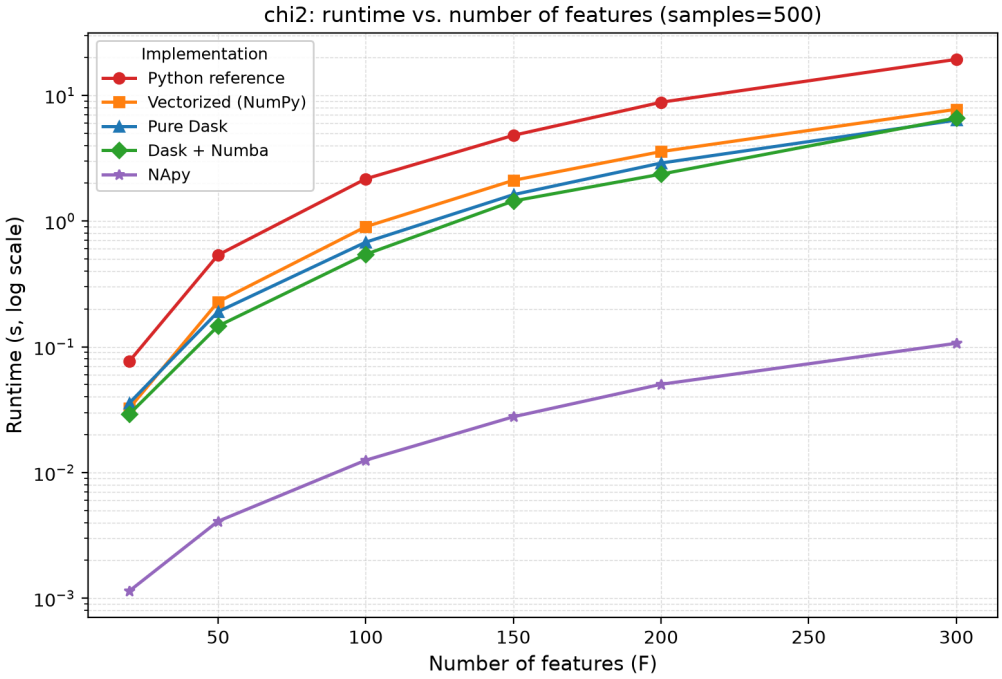

Figure 1: Chi-squared test: runtime as a function of the number of variables (features), with the number of samples fixed at 500. Runtime axis on log scale. All five configurations (Python reference, vectorized NumPy, pure Dask, Dask+Numba, and NApy) are shown.

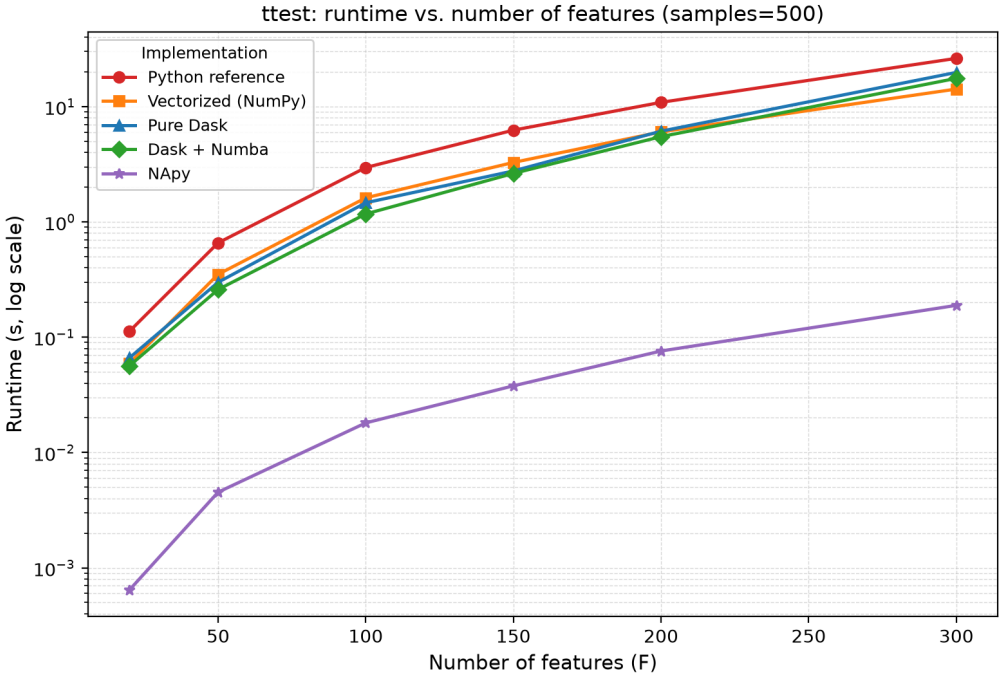

Figure 2: *t*-test: runtime as a function of the number of variables (features), with the number of samples fixed at 500. Runtime axis on log scale.

2.2 Runtime scaling with number of samples

Figures 3 and 4 fix the number of variables at 100 and vary the sample count from 100 to 2000. NApy's runtime grows only modestly with sample count, remaining under 0.07 s across the full range for both tests. The Python reference implementation shows clear growth and becomes the slowest configuration once sample count exceeds roughly 250–500 samples. The vectorized, Dask, and Dask+Numba configurations grow more slowly than the reference implementation but plateau at broadly similar runtimes to one another. On our 4-core test machine, sample count alone was not sufficient to expose large differences between the three parallelization strategies at this fixed, moderate number of variables.

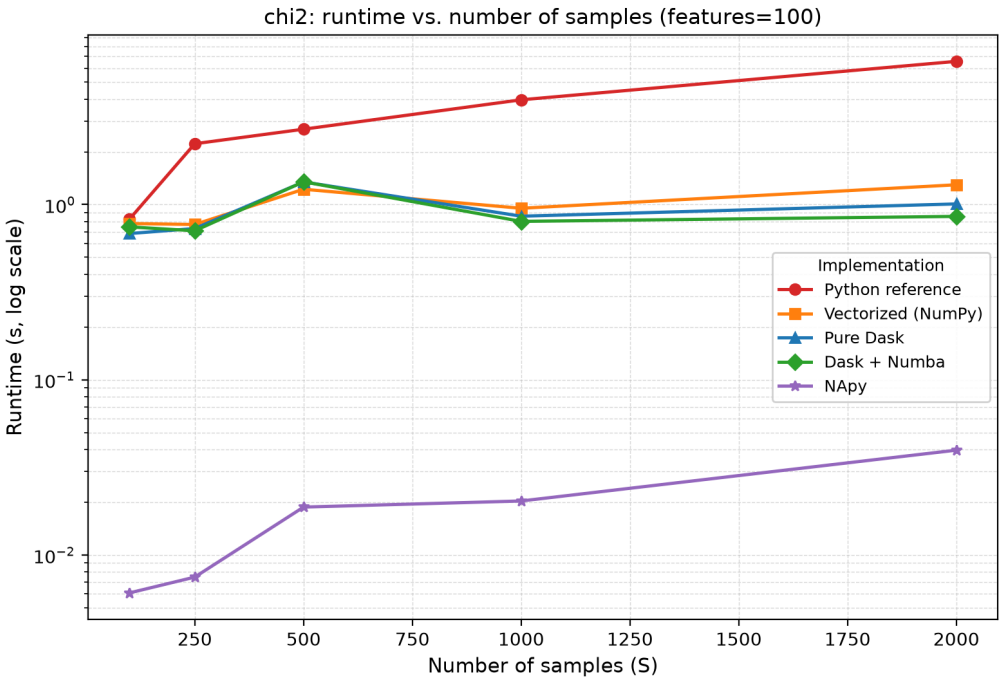

Figure 3: Chi-squared test: runtime as a function of the number of samples, with the number of variables fixed at 100. Runtime axis on log scale.

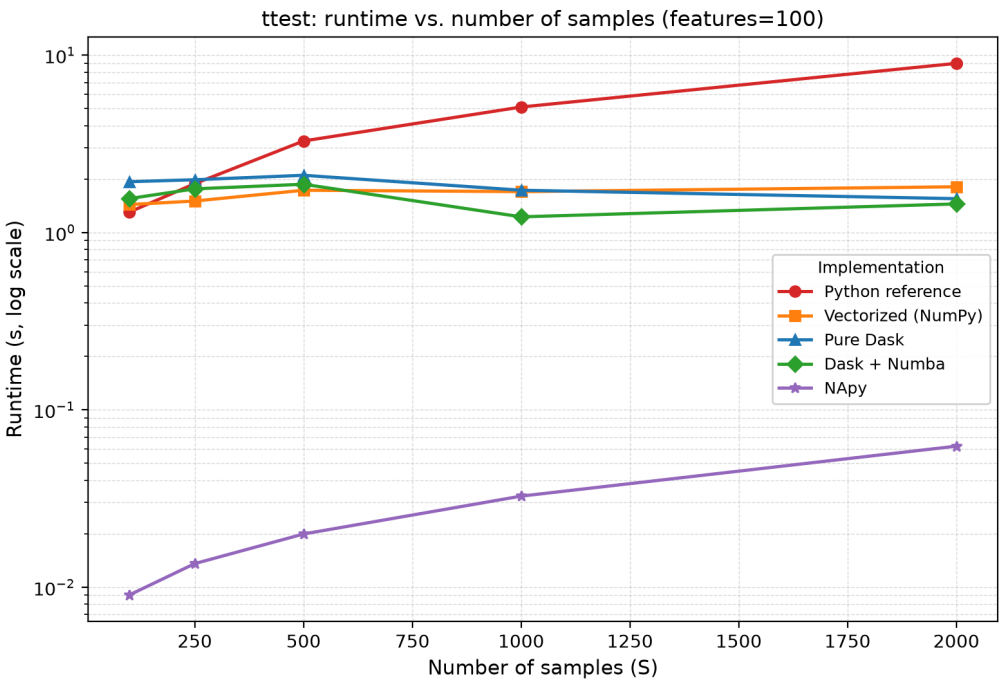

Figure 4: *t*-test: runtime as a function of the number of samples, with the number of variables fixed at 100. Runtime axis on log scale.

2.3 Effect of Dask task granularity (workload per process)

Figures 5 and 6 fix the problem size (200 features, 500 samples) and vary the number of variable pairs dispatched per Dask task from 5 to 400. Runtime decreases sharply as workload increases. For the chi-squared test, pure Dask improves from 30.60 s (workload$=5$) to 1.90 s (workload$=200$) before leveling off, while Dask+Numba improves further, from 30.33 s to 1.38 s at workload$=400$, a $\sim$22-fold improvement. The *t*-test workload sweep shows the same pattern: Dask+Numba improves from 70.55 s (workload$=5$) to 2.35 s (workload$=400$), a $\sim$30-fold improvement. In both tests, Dask+Numba matches or outperforms pure Dask at every workload once tasks are large enough to amortize Numba's JIT-compilation overhead, roughly workload ≥ 25. At very small workloads, the two configurations perform similarly, since scheduling overhead dominates regardless of the per-pair computation method.

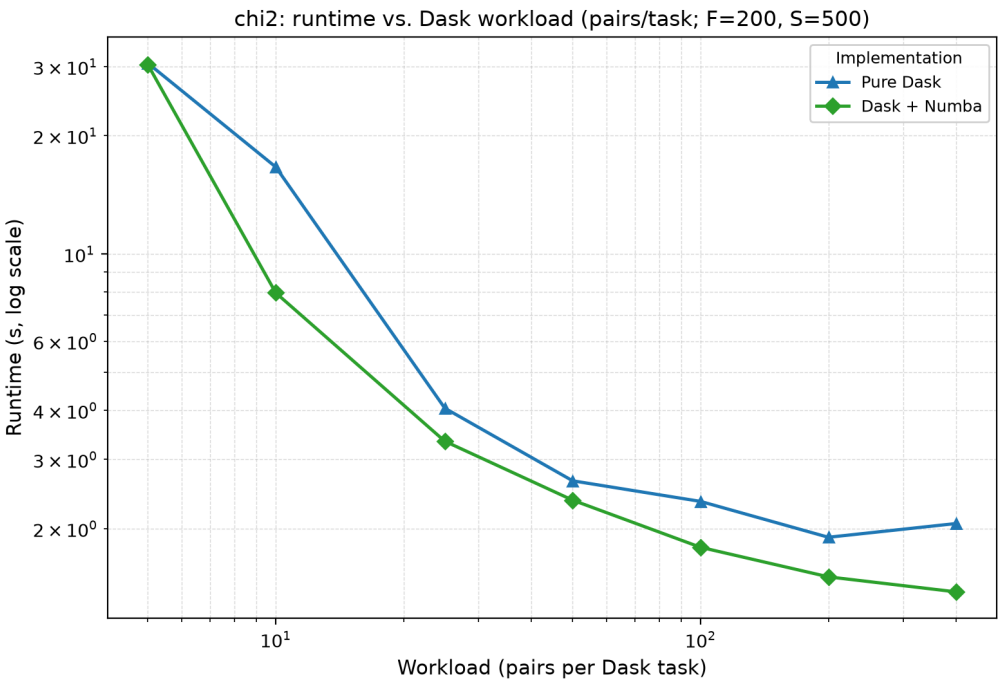

Figure 5: Chi-squared test: runtime as a function of the Dask workload (variable pairs dispatched per task), for a fixed problem size of 200 features and 500 samples. Only the pure Dask and Dask+Numba configurations are shown, since workload is a parameter specific to Dask task scheduling.

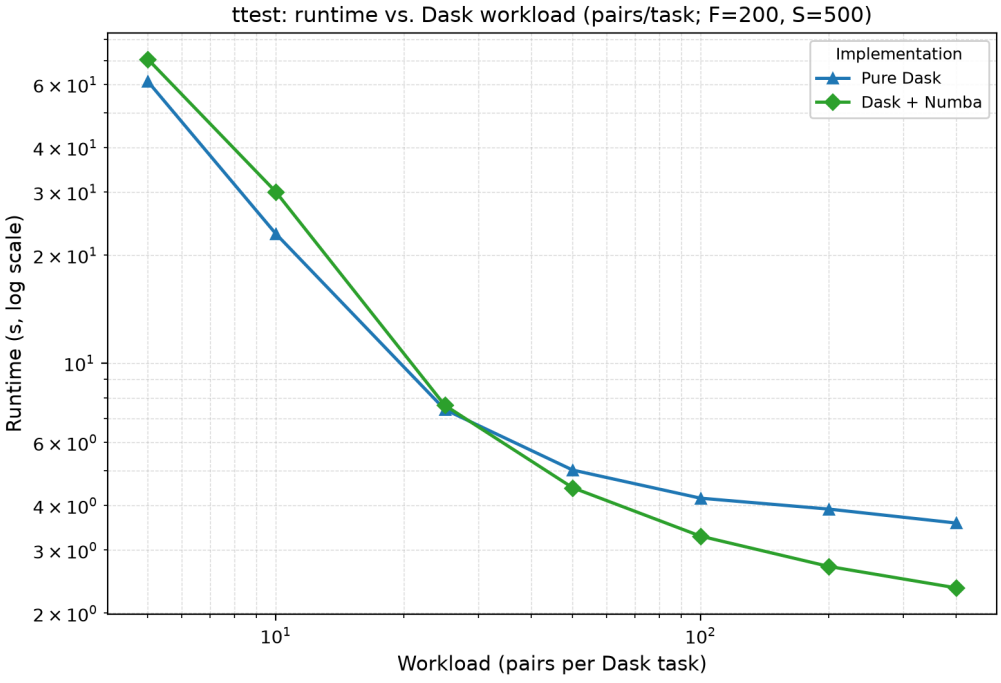

Figure 6: *t*-test: runtime as a function of the Dask workload (variable pairs dispatched per task), for a fixed problem size of 200 features and 500 samples.

Table 1: Runtime (seconds) at the largest tested problem size (300 features, 500 samples), sorted fastest to slowest.

Implementation	Chi-squared (s)	*t*-test (s)
NApy	**0.11**	**0.19**
Dask	6.35	19.80
Dask + Numba	6.60	17.59
Vectorized (NumPy)	7.74	14.20
Python reference	19.31	26.22

3 Discussion

Across both statistical tests, our results show that vectorization alone gives a substantial speed-up over naive Python looping, and that combining Dask-based parallelization with Numba JIT compilation gives the best runtime among our four self-implemented configurations. Even so, our best configuration remains roughly two orders of magnitude slower than NApy's compiled C++/OpenMP backend. The granularity of parallel task scheduling has a large effect on Dask's performance: fine-grained tasks with few variable pairs per task are dominated by scheduling overhead, while coarser-grained tasks let the compiled Numba kernels dominate runtime instead, which improves throughput substantially.

Our findings are consistent with those reported in the original NApy publication [1], which found that a compiled C++/Numba backend with OpenMP parallelization outperforms naive Python-based approaches by orders of magnitude on both simulated and real biomedical data, with speed-ups reaching over 1000× against the fastest competitor in some configurations. Our project adds a more granular view of this gap. Rather than comparing NApy only against a single naive baseline, we isolate the individual contribution of vectorization, task-based parallelism (Dask), and JIT compilation (Numba), and find that even a well-optimized combination of Python-ecosystem parallelization tools does not close the performance gap to a fully compiled implementation. This fits with a known limitation of Python-orchestrated parallelism: each Dask task incurs process-level scheduling and (de)serialization overhead that a compiled, thread-parallel (OpenMP) implementation avoids entirely. NApy's own benchmarks were run on a compute-cluster server with two 16-core Intel Xeon Silver 4216 CPUs (64 threads total), while our benchmarks ran on a single 4-core consumer laptop. This hardware difference likely accounts for part, though not all, of the performance gap we observe.

3.1 Limitations

Our benchmarks were run on a single 4-core consumer laptop, which limits the achievable parallel speed-up from Dask compared to what a many-core server such as the one used in the original NApy benchmarks might show. This probably explains why the runtime differences between pure Dask and vectorized NumPy were relatively modest in the samples-scaling experiment (Figures 3, 4): with only 4 physical cores, there is limited room for parallel gains. Our Dask+Numba *t*-test implementation also currently only supports the equal-variance (Student's) *t*-test; Welch's *t*-test (`equal_var=False`) is not yet implemented for this configuration, although it is supported in the other three. All benchmarks reported here use Student's *t*-test, matching NApy's default setting, so this gap does not affect the validity of the reported comparisons. It does represent a feature-parity gap that would need to be closed for full interchangeability of the four configurations.

3.2 Outlook

A useful next step would be running this comparison on a multi-core server, to better characterize Dask's parallel scaling behavior at higher core counts, closer to the conditions under which NApy itself was benchmarked. It would also help to profile where the remaining runtime gap between Dask+Numba and NApy actually comes from: Python-level task scheduling overhead, inter-process data serialization, or

remaining differences in the compiled numerical kernels themselves. Answering that would give a clearer picture of the realistic upper bound on performance achievable within a pure Python/Dask ecosystem.

In summary, we implemented and benchmarked four configurations of the pairwise chi-squared test of independence and Student's *t*-test, and compared their runtime scaling and parallel task granularity against the NApy package. Dask+Numba consistently gave the best runtime among our own implementations, particularly at larger per-task workloads, but all four configurations stayed substantially slower than NApy's compiled backend. These results agree with the performance advantages reported in the original NApy publication and show, concretely, what a Python-based parallelization strategy can and cannot achieve for a computationally intensive, embarrassingly parallel statistical workload.

4 Methods

4.1 Statistical tests

Chi-squared test of independence. For an integer-encoded categorical data matrix of shape (F, S), we compute the chi-squared statistic and two-sided P-value of the test of independence for every pair of variables. The contingency table for each pair is built from pairwise-complete observations, following Equation (1): samples missing in either variable of a pair are excluded for that pair only.

Student's *t*-test. For a binary variable matrix (0/1-encoded) and a continuous variable matrix of matching sample dimension, we compute the two-sample, equal-variance *t*-statistic and two-sided P-value for every (binary, continuous) variable pair. Samples are split into two groups by the binary variable's value, again with pairwise-complete missing-value handling.

Both tests follow the data convention and default parameterization (`equal_var=True` for the *t*-test) used by NApy, which allows a direct comparison.

4.2 Implementations

We implemented four configurations for each test.

1. **Python reference**: a naive implementation using explicit nested Python loops over all variable pairs, and within each pair, over categories or samples, calling `scipy.stats` [3] for the underlying test computation. This is our correctness baseline and worst-case performance bound.

2. **Vectorized (NumPy)**: replaces per-pair inner loops with vectorized NumPy [2] operations, for example `np.histogram2d` for contingency tables and boolean masking for group splitting, computing the test statistic directly via array arithmetic.

3. **Pure Dask**: variables are partitioned into row-blocks (the workload per task). A `dask.delayed` [5] task is created for each block-pair combination, using the same vectorized routine as configuration 2 internally.

4. **Dask + Numba**: the same block-based Dask task graph as configuration 3, but the per-block computation kernel is compiled just-in-time with Numba [4], combining compiled-code speed with multi-process parallelism.

4.3 Correctness verification

We verified all four configurations produce numerically identical chi-squared/*t* statistics and P-values, within floating-point tolerance, using an automated `pytest` test suite (23 tests). This included direct comparison against `scipy.stats` ground truth on hand-constructed examples and against the real NApy package's output on simulated data.

4.4 Benchmarking design

We simulated categorical, binary, and continuous data matrices of controlled size with a fixed missing-value ratio. We performed two independent sweeps for each test: (i) varying the number of variables (features) with sample count fixed at 500, and (ii) varying the number of samples (100–2000) with variable count fixed at 100. Running these as two separate sweeps avoids conflating the two axes, which would happen if we averaged over a full features $\times$ samples grid. A third experiment fixed both dimensions (200 features, 500 samples) and varied the Dask block size (number of variable pairs dispatched per task) from 5 to 400, to isolate the effect of task granularity on the Dask and Dask+Numba configurations. We ran all benchmarks on a 4-core consumer laptop (Intel i7-11165G7), timing each configuration with Python's high-resolution performance counter and averaging over multiple repeats.

5 Code availability

The source code, tests, and benchmarking scripts for this project are publicly available on GitHub at `https://github.com/Aferdous75/BIONET`, with a permanent archived release available at DOI: 10.5281/zenodo.21643971.

6 Data availability

This project uses only synthetically simulated data (categorical, binary, and continuous matrices generated with fixed random seeds), and no external or third-party dataset was used. The simulation code (`simulate.py`) is included in the linked GitHub repository (Section 5), so all data underlying the reported results can be exactly regenerated by running the benchmarking scripts with the documented parameters and seeds.

References

[1] Woller, F., Arend, L., Fuchsberger, C., List, M., & Blumenthal, D. B. (2025). NApy: Efficient Statistics in Python for Large-Scale Heterogeneous Data with Enhanced Support for Missing Data. *GigaScience*, giaf140. https://doi.org/10.1093/gigascience/giaf140

[2] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

[3] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261–272. https://doi.org/10.1038/s41592-019-0686-2

[4] Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based Python JIT Compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC* (LLVM '15), Article 7. https://doi.org/10.1145/2833157.2833162

[5] Rocklin, M. (2015). Dask: Parallel Computation with Blocked algorithms and Task Scheduling. In *Proceedings of the 14th Python in Science Conference*, 126–132. https://doi.org/10.25080/Majora-7b98e3ed-013

A AI usage declaration

In the boxes below, I specify which AI tool I used, to what extent, and for which task.

Task: Brainstorming and concept development

I used AI tools to discuss the project requirements and clarify the overall approach before starting the implementation.

Task: Literature review and analysis

I used AI tools to help understand the NApy documentation and related paper, and to summarize some background information.

Task: Methodology

I used AI tools to discuss possible implementation approaches and the overall benchmarking procedure.

Task: Coding

I used AI tools to explain programming concepts, suggest code improvements, and help debug some implementation issues. The final code was written, tested, and modified by me.

Task: Data collection, generation, and analysis

I used AI tools to discuss the data generation process and to help understand the benchmark results.

Task: Interpretation and validation

I used AI tools to discuss the interpretation of the benchmark results and compare them with the reference implementation.

Task: Structuring and planning the text

I used AI tools to suggest a suitable structure for the report based on the provided template.

Task: Text generation

I used AI tools to improve the wording of some sections. The final text was reviewed and edited by me..

Task: Translating text

I did not use any AI tools for this task.

Task: Review and editing of the text

I used AI tools to check grammar, improve clarity, and suggest minor revisions.

Task: Bibliography management

I used AI tools to check the formatting of some references.

Task: Other activities

I used AI tools to set up the Python environment, run the correctness test suite and benchmarking scripts.

B Declaration of academic integrity

I hereby declare that I have authored this work independently and without the use of any sources or assistance other than those specifically cited. All content taken verbatim or paraphrased from other sources has been clearly identified as such. Furthermore, I certify that this work has not been submitted, in its current or a similar form, for any other examination. All usage of AI tools has been fully disclosed in Appendix A, and I take full responsibility for all content generated with the help of AI. I am aware that any false or incomplete declarations given here or in Appendix A may be treated as attempts at deception and could thus lead to the failure of this examination. I am also aware that inability to provide in-depth technical explanations for any content contained in this work or in the underlying source code may be treated as evidence for illicit AI usage or misappropriation of the work of others and could thus lead to the failure of this examination.

Arafa Ferdous

Erlangen, 28 July 2026